CPSC 314
Assignment 4: Textures and Shadows

Due 11:59PM, March 28, 2026

1 Introduction

In this assignment you will be learning about different uses of textures. You will implement a skybox, give your favourite armadillo a new, dazzling look, and make it cast shadows. And you will meet a new protagonist, Shay D. Pixel, a SIGGRAPH mascot, explore physically based rendering further and have a taste of image-based lighting.

1.1 Getting the Code

Assignment code is hosted on the UBC Students GitHub. To retrieve it onto your local machine navigate to the folder on your machine where you intend to keep your assignment code, and run the following command from the terminal or command line:

```
git clone https://github.students.cs.ubc.ca/CPSC314-2025W-T2/a4-release.git
```

1.2 Template

- The file `A4.html` is the launcher of the assignment. Open it in your preferred browser to run the assignment, to get started.
- The file `A4.js` contains the JavaScript code used to set up the scene and the rendering environment.
- The folder `glsl/` contains the vertex and fragment shaders.
- The folder `js/` contains the required JavaScript libraries. You do not need to change anything here.
- The folder `gltf/` contains geometric models to be used in the scene.
- The folder `images/` contains the texture images used.

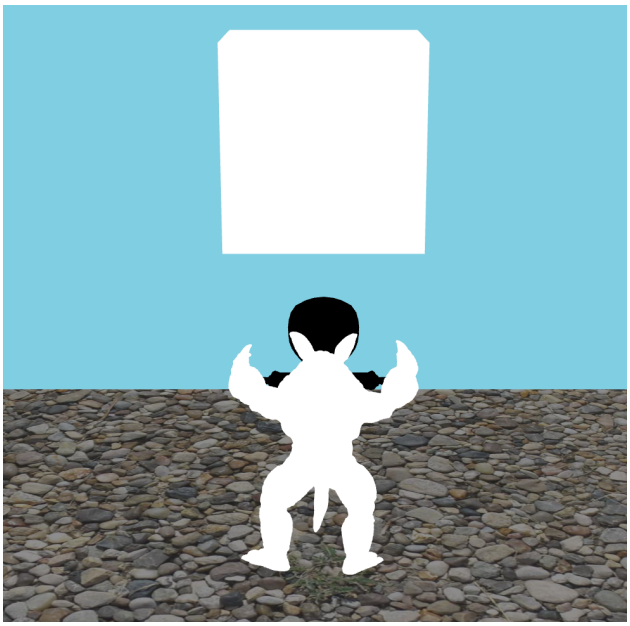


Figure 1: Initial configuration.

2 Work to be done (100 points)

Here we assume you already have a working development environment which allows you to run your code from a local server. If you do not, check out instructions from Assignment 1 for details. The initial scene should look as in Fig. 1.

2.1 Part 1: Required Features (85 points)

a. **(10 points)** Texture Mapping with ShaderMaterial.



Figure 2: Question a: Texture mapping with ShaderMaterial.

In this part you will implement texture mapping for Shay D. Pixel using shaders. You are provided with color texture `images/Pixel_Model_BaseColor.jpg`. The geometric model `gltf/pixel_v4.glb` is in GLB format which you have seen in previous assignments, and it has vertex UV coordinates baked in.

Your task is to pass the textures (as a uniform) to the fragment shader (`shay.fs.glsl`), use the right UV coordinates to sample a color from the texture, and then use the sampled color and the light intensity (which has been computed for you in `shay.fs.glsl`) to calculate the final fragment color. The result should look close to what is shown in Fig. 2.

Hint 1: The texture is flipped on the y-axis. Take this into consideration when you assign the UV coordinates.

b. **(10 points)** Skybox.



Figure 3: Question b. Skybox.

A skybox is a simple way of creating backgrounds for your scene with textures. We have provided six textures under `images/envmap/`. You will implement a skybox using cube environment mapping as discussed in class. Specifically, in `A4.js` load the six textures to `skyboxCubemap` in a proper order; then set this cubemap to the background of the scene. In Three.js you can add a skybox background by setting `scene.background` to `CubeTexture`: see `https://threejs.org/manual/?#en/backgrounds`. The final result should look close to what is shown in Fig. 3 (just the background).

Hint 1: Make sure you load the six textures in the correct order. You can refer to `https://threejs.org/docs/#CubeTextureLoader` for details.

c. **(25 points)** Shiny Armadillo and Cube.



(a) Look1 (b) Look2

Figure 4: Two views of the environment mapping effects.

Another interesting use of `samplerCubes` is **environment mapping**. This can be used to make the same, boring armadillo highly reflective, like a mirror. For this part, complete the shaders `envmap.vs.glsl` and `envmap.fs.glsl` to implement a basic reflective environment map shader. The result should be close to what is shown in Fig. 4. You can use the same cube texture `skyboxCubemap` in your shaders which is passed as a `samplerCube` uniform like before, as well as the same `texture()` function, but pay attention to use the correct `vec3`, described in class and in the textbook, to retrieve the texture color.

After finishing the shaders, apply this env map material to both the armadillo and the cube in the scene. You should be able to see the skybox reflected on their surfaces as shown in Fig. 4. This cube is there for you to debug and us to grade easier. Please leave it in the scene.

Hint 1: Note the cube texture sampling in GLSL uses left-handed coordinate system, and you might need to modify some values in the shaders to address that. See `https://threejs.org/docs/#CubeTextureLoader` for more information.
Hint 2: Think about how the armadillo (or cube) should look from various directions.

How should it look from the bottom? The top? Rotate the camera around the armadillo to help you debug.

d. **(30 points)** Shadow Mapping.



(a) PCF smoothing (b) Non smoothed

Figure 5: Question d: Shadow Mapping.



Figure 6: Question d: Depth Map.

Shadows are a tricky part of computer graphics, since it is not easy to figure out which parts of a scene should be cast in shadow. There are many techniques to create shadows (raytracing, shadow volumes, etc.); we will use shadow mapping in this assignment. Shadow mapping is all about exploiting the z-buffer: a shadow map is rendered in an off-screen frame buffer by projecting the scene from the perspective of a light source,

giving us a depth-like value at each fragment along rays of the light source. Your task is implementing shadow mapping, so that smooth shadows of the armadillo and Shay can be casted onto the floor: see Fig. 5. You can switch between scene views by pressing key 1, 2, and 3 for the scene, depth scene, and shadowed scene respectively. Scene 2 and 3 are for you to implement; the result from rendering scene 2 is shown in Fig. 6. We have listed the steps for you to follow:

1. Start by creating the shadow map, which is the depth map when viewed from the light source. Add appropriate object(s) to the provided `shadowScene`, do a first pass render to a `WebGLRenderTarget` to create the depth map, and finally visualise this using the provided `postScene` (short for "post processing scene"). Your primary job will be to pass the appropriate textures between render targets and to implement the render shaders (`render.vs.glsl` and `render.fs.glsl`). You will find the API docs useful: `https://threejs.org/docs/#api/en/renderers/WebGLRenderTarget`.
2. Next, use the depth map to project the shadows onto the floor. This will involve modifying the floor's shader code to check whether a fragment is in shadow or outside. You can do this by transforming the fragment's position to "light space" (i.e., in the shadow camera's coordinate frame), and using that to compute the appropriate texture coordinate in the shadow map. Then compare the depth of the fragment to the value stored in the shadow map. After this step you should see casted shadows similar to ones shown in Fig. 5.
3. Lastly, you will smooth the shadows by using percentage closer filtering (PCF), in the floor shader code. PCF is a shadow anti-aliasing technique which reduces 'jaggies' by replacing the binary in/not-in shadow calculation of a pixel, with a calculation that instead checks if a pixel and its neighbours are in shadow, and 'shadows' the pixel according to the fraction that are.

e. **(10 points)** Image-based Lighting.

Image-based lighting (IBL) is a rendering technique which requires capturing real-world light information as a High Dynamic Range (HDR) image. This image is then used to calculate the lighting for objects in the scene. IBL can create incredibly realistic simulations of real world lighting, especially when used with physically based materials as we discussed in class. You will set up a scene in three.js to use IBL. You can enter the IBL scene by pressing key 4.



Figure 7: Question e: Completed IBL scene.

Even though this can be complicated, we have made the problem as simple as possible, so that you can get a flavor for how to use these techniques without getting bogged down. In A3 you have already learned how to load a scifi-themed helmet and apply PBR when texturing it. For this assignment you just need to complete `helmetMaterial` by setting the material's environment map property after loading the HDR texture. The result should resemble what is shown in Fig. 7.

2.2 Part 2: Feature Extension (15 points)

In this part, your task is to extend the assignment to add features of your own choosing. The goal is to encourage you to explore the capabilities of Three.js and WebGL, and more importantly, unleash your creativity! The extension has to be non-trivial to receive credit, but for full credit, the features have to be truly creative. See Sec. 4.2 for grading expectations. A small number of exceptional extensions will be selected from this group and shown in class, with the student's permission. You can check Assignment 1's spec document for ideas to get you started, and feel free to come up with your own ideas.
First duplicate your current work into a new directory named 'part2', then implement your feature in this new directory. Write a brief description of your feature in the README file. You will be graded on both how it works and how well you can explain your feature.

3 Submission Instructions

3.1 Directory Structure

Your submission should contain two subdirectories - the first should be named 'part1' and should contain all parts except the feature extension; the second subdirectory should be named 'part2' and should contain your feature extension. Under the root directory of your assignment, please add both subdirectories including all the source files and everything else required to run each part in the respective folder. Do not create more sub-directories than the ones already provided.

You must also write a clear `README.txt` file which includes your name, student number, and CWL username, instructions on how to use the program (keyboard actions, etc.) and any information you would like to pass on to the marker. Place the file under the root directory of your assignment.

3.2 Submission Methods

Please compress everything under the root directory of your assignment into `a4.zip` and submit it on Canvas. You can make multiple submissions, but we will grade only the last one.

4 Grading

4.1 Face-to-face (F2F) Grading

Submitting the assignment is not the end of the game; to get a grade for the assignment, you must meet face-to-face with a TA in a 12-min slot during or outside lab hours, on Zoom, to demonstrate that you understand how your program works; Grading slots and detailed instructions on how to sign up and conduct F2F will be announced on Canvas and on Piazza. During the meeting, the TA will (1) ask you to run your code and inspect the correctness of the program; (2) ask you to explain parts of your code; (3) ask you some questions about the assignment, and you will need to answer them in a limited timeframe. The questions will mostly be based on ThreeJS or WebGL concepts that you must have come across while working on the assignment; but also you may get conceptual questions based on lecture materials that are relevant to the assignment, or technicalities that you may not have thought about unless you really "digged deep" into the assignment. But no need to be nerveous! We evaluate your response based mainly if not entirely on the coherence of your thoughts, rather than how complete or long your response is: e.g. if the full answer to a question includes A+B+C+D and you mentioned A+B only in the provided time, but your thread of thought is logical, then you may still get full marks.

4.2 Point Allocation

All questions before Feature Extension have a total of 85 points. The points are warranted based on

- The functional correctness of your program, i.e., how visually close your results are to expected results;
- The algorithmic correctness of your program, e.g., applying transformation matrices in the right order;
- Your demonstrating your understanding of the code with your answers to TAs' questions during face-to-face (F2F) grading.

The feature extension grade is based on creativity and sophistication of your submission. We warrant partial marks on four scales:

- 15: Outstandingly creative and sophisticated feature extension. But note that only a very small number of extensions may get this score, and we cannot guarantee that if you implement one of the suggestions as is then you will definitely get full marks;
- 10: Average creativity and sophistication, e.g. adding a few interesting objects into the scene and animating, deforming the objects in visually appealing ways;
- 5: Basic creativity and sophistication, e.g. adding a few static objects into the scene, changing the orb's shape in simple ways;
- 0: Trivial or no feature extension, e.g. just changing the color of the orb.

4.3 Collaboration Policy

Submissions must be individual, but you are encouraged to discuss the assignment with your classmates, and you are allowed to use generative AI tools, e.g. ChatGPT, Claude to help with constructing your solution. **In the `README.txt` file, include names of the people you discussed the assignment with, websites you acquired information from, and if you used generative AI tools, include the names of the tools you used.** And you must understand your code and be able to explain them during F2F grading. If you cannot demonstrate at least a basic understanding of your code during face-to-face grading, you may receive a zero for the assignment—even if your code runs perfectly. Hiring a tutor to write your assignment, or copy-pasting others' code without proper referencing in `README.txt` are considered as plagiarism, and will be reported to the dean's office.

4.4 Penalties

Aside from penalties from incorrect solution or plagiarism, we may apply the following penalties to each assignment:

Late penalty. You are entitled up to three grace (calendar) days in total throughout the term. No penalties would be applied for using them. However once you have used up the grace days, a deduction of 10 points would be applied to each extra late day. Note that

1. The three grace days are given for all assignments, **not per assignment**, so please use them wisely;

2. We check the time of your last submission to determine if you are late or not.

No-show penalty. Please sign up for a grading slot on the provided sign-up spreadsheet (link will be posted later on Piazza) before the submission deadline of the assignment, and show up to your slot on time. A 10-point deduction would be applied to each of the following circumstances:

1. Not signing up a grading slot before the sign-up period closes. Unless otherwise stated, the period closes at the same time as the submission deadline.

2. Not showing up at your grading slot.

If none of the provided slots work for you, or if you have already missed your slot, follow instructions outlined in this Piazza post to get graded after the F2F grading period ends. Also, please note that

1. you'll need to explain to your TA why you're getting graded late, and may be asked to present documents to justify your hardship. The TA may remove the no-show penalty as long as he/she deems the justification to be reasonable.

2. In the past some students reported that their names disappeared mysteriously due to technical glitches, and the spreadsheet's edit history has no trace of it. So double check that your name is on the sign-up sheet after you sign up by refreshing the page.